
UTEC | TECNÓLOGO EN INFORMÁTICA

Taller de Sistemas de Información .NET

Laboratorio

Visión, realidad y requerimientos

Plataforma Multioperador de Distribución de Última Milla

Edición 2026

Docente: Nicolás Escobar Di Camillo

Presentación de la letra	jueves 17 de septiembre de 2026
Entrega de análisis y diseño	domingo 4 de octubre de 2026, 23:59 h
Primer monitoreo	jueves 8 de octubre de 2026, 19:00 h
Entrega final	domingo 15 de noviembre de 2026, 23:59 h
Defensa	jueves 19 de noviembre de 2026

Índice

1. Información administrativa	1
2. Visión del laboratorio	1
3. Descripción de la realidad	2
4. Alcance: aplicaciones a construir	3
5. Requerimientos funcionales	3
6. Requerimientos no funcionales obligatorios	5
7. Requerimientos opcionales	8
8. Entregas y cronograma	9
9. Evaluación	11

1. Información administrativa

El laboratorio comienza el jueves 17 de septiembre de 2026 y culmina con la defensa el jueves 19 de noviembre de 2026.

Ambas entregas se realizarán a través del Moodle del curso. La entrega de análisis y diseño vence el **domingo 4 de octubre a las 23:59 h.** La entrega final, que incluye la documentación, la presentación y el código fuente, vence el **domingo 15 de noviembre a las 23:59 h.**

1.1 Conformación de los equipos

El trabajo se realiza en equipos de 4 o 3 integrantes, teniendo estos últimos el alcance reducido que se indica expresamente más adelante.

1.2 Dedicación estimada

La dedicación prevista es de 12 horas semanales por integrante durante aproximadamente 9 semanas, lo que representa del orden de 430 horas de trabajo por equipo. El alcance de esta letra está dimensionado para esa carga. Cada equipo deberá llevar un registro de horas discriminado por integrante y por tipo de actividad (análisis, desarrollo, infraestructura, pruebas, documentación e investigación), que se entregará junto con la documentación final.

1.3 Instancias de seguimiento

Los monitoreos se realizarán los días jueves a las 19:00 h, a partir del jueves 8 de octubre, y la asistencia será obligatoria para todos los integrantes. Cada monitoreo tendrá un objetivo verificable, detallado en la sección 8: no se trata de exponer avances, sino de **demostrar en ejecución** lo comprometido para esa fecha.

2. Visión del laboratorio

Este laboratorio propone construir un sistema de información empresarial completo, de extremo a extremo, sobre la plataforma .NET. El objetivo no es únicamente entregar un producto que funcione, sino que cada integrante desarrolle y pueda demostrar las competencias que se le exigen a un profesional que se incorpora a un equipo de desarrollo: tomar decisiones de arquitectura fundamentadas, conociendo las alternativas que descartó y las consecuencias de lo que eligió; diseñar software configurable, que se adapte a clientes distintos sin modificar el código; comprender y operar los mecanismos propios de los sistemas distribuidos, como el caché, la mensajería asíncrona, el escalado horizontal y la tolerancia a fallos; trabajar con infraestructura en la nube de forma

reproducibles; investigar tecnologías que no se dictaron en clase y justificar su adopción o su descarte con evidencia; y explicar, defender y modificar el propio trabajo frente a terceros.

Sobre el uso de asistentes de inteligencia artificial. Su uso está permitido y forma parte del ejercicio profesional actual. Lo que se evalúa en este laboratorio, sin embargo, es la comprensión que cada estudiante pueda demostrar, no el código entregado. Para eso se aplicarán los mecanismos de la sección 9: calificación individual diferenciada, bitácora de uso de IA, demostraciones en vivo con escenarios no anunciados, incorporación de un cambio de requerimientos en pleno desarrollo y defensa oral del código propio. Un componente que su responsable no pueda explicar se considerará no entregado a los efectos de su calificación individual.

3. Descripción de la realidad

3.1 La situación actual

El crecimiento sostenido del comercio electrónico dejó a los operadores logísticos pequeños y medianos trabajando con herramientas que no acompañan el volumen que manejan. Los comercios les envían los pedidos por planilla de cálculo o por correo electrónico. El despachador arma las hojas de ruta a mano la noche anterior, repartiendo los envíos entre los repartidores según su conocimiento del territorio. El repartidor sale a la mañana con una lista impresa y un grupo de mensajería instantánea como único canal de coordinación.

Durante la jornada nadie sabe con certeza dónde está cada paquete. Cuando el comprador llama al comercio preguntando por su envío, el comercio llama al operador, el operador llama al repartidor, y la respuesta llega cuando llega. Cuando una entrega falla no queda registro confiable del motivo, ni evidencia de que el repartidor concurrió al domicilio, lo que genera reclamos que después nadie puede dirimir. Las devoluciones se acumulan en el depósito sin trazabilidad. Cada comercio pide información en su propio formato y ninguno puede consultar por sus propios medios el estado de sus envíos.

A esto se agrega que la conectividad móvil en las zonas de reparto es irregular: hay sectores del área metropolitana, y buena parte del interior, donde el repartidor trabaja horas sin señal utilizable.

3.2 El sistema propuesto

Se propone construir una plataforma de gestión de distribución de última milla, ofrecida como servicio a múltiples operadores logísticos. Un único despliegue atiende a varios operadores, cada uno con su marca, sus zonas de cobertura, su cuadro tarifario, sus reglas de operación y sus usuarios, sin que ninguno pueda acceder a dato alguno de otro.

La plataforma debe permitir que los comercios carguen sus envíos y sigan su estado de forma autónoma, que el operador planifique y controle la operación diaria, que el repartidor trabaje en la calle con o sin conectividad, y que el destinatario final conozca el estado de su paquete sin necesidad de llamar a nadie.

3.3 Actores

El modelo de actores tiene tres niveles, y tratarlo correctamente es una de las decisiones de diseño centrales del laboratorio.

El **operador logístico** es el inquilino (*tenant*) de la plataforma: contrata el servicio y opera con independencia de los demás operadores. Dentro del operador trabajan el administrador, que configura la operación, las zonas, las tarifas, los usuarios y las reglas de negocio; el despachador, que planifica las rutas del día, asigna repartidores, monitorea la jornada y resuelve incidencias; el repartidor, que ejecuta la hoja de ruta en la calle; y el operario de depósito, que recibe, clasifica y despacha bultos.

El **comercio** es cliente del operador logístico. Tiene sus propios usuarios, carga envíos y accede exclusivamente a su información. Un mismo comercio puede operar con más de un operador logístico.

El **destinatario** es el consumidor final. No pertenece a ningún operador, no tiene cuenta en la plataforma y accede al seguimiento de su envío mediante un enlace público que recibe por correo

electrónico o mensajería.

3.4 Entidades de referencia

El modelado detallado es responsabilidad de cada equipo. A modo orientativo, sin que la lista sea exhaustiva ni obligatoria, se sugieren: Operador, Comercio, Usuario, Envío, Bulto, Destinatario, Dirección, Zona de cobertura, Franja horaria, Ruta, Parada, Vehículo, Repartidor, Evento de envío, Intento de entrega, Prueba de entrega, Motivo de no entrega, Incidencia, Devolución, Tarifa, Liquidación, Suscripción de avisos y Notificación.

4. Alcance: aplicaciones a construir

La solución se compone de cinco aplicaciones desplegables, más los servicios de soporte que correspondan.

Backoffice del operador (ASP.NET Core Razor Pages). Centro de operaciones del operador logístico: administración de comercios, usuarios y permisos; definición de zonas de cobertura y franjas horarias; configuración del cuadro tarifario y de las reglas operativas; recepción y clasificación de bultos en depósito; armado, despacho y seguimiento de hojas de ruta; gestión de incidencias, reprogramaciones y devoluciones; tablero de operación en vivo; y reportes de gestión.

Portal del comercio (Blazor). Autogestión para los comercios clientes: alta de envíos en forma individual, por importación de archivo y por API; impresión de etiquetas; consulta y seguimiento de sus envíos; gestión de devoluciones; configuración de los avisos automáticos que desea recibir en sus propios sistemas; y consulta de su cuenta corriente y liquidaciones.

Seguimiento público (Blazor, sin autenticación). Página que abre el destinatario con el enlace que recibe: estado actual del envío, ventana horaria estimada, historial de eventos y solicitud de reprogramación. Es el componente con mayor volumen de tráfico y el único expuesto sin autenticación, de modo que concentra exigencias particulares de caché, limitación de tasa y protección de datos personales.

Aplicación del repartidor (.NET MAUI). Herramienta de trabajo en la calle: descarga de la hoja de ruta del día, escaneo de bultos al cargar el vehículo, navegación entre paradas, registro de cada entrega o intento fallido con su evidencia, gestión de devoluciones y rendición al regresar al depósito. Debe operar de forma completa sin conectividad.

Servicio de procesamiento en segundo plano (worker .NET). Aplicación desplegada de forma independiente de la API, que consume mensajes de la cola y ejecuta el procesamiento de los eventos de seguimiento provenientes de la aplicación móvil, el envío de notificaciones a destinatarios, la entrega de avisos automáticos a los sistemas de los comercios, el recálculo de indicadores de cumplimiento y el cierre diario de la operación.

5. Requerimientos funcionales

Se deberán cubrir todos los casos que se desprenden de la descripción de la realidad. Se detallan a continuación los requerimientos mínimos.

5.1 Administración y configuración del operador

1. Alta, baja, modificación y consulta de comercios clientes y de sus usuarios, con asignación de perfiles y permisos.
2. Definición de zonas de cobertura y de las franjas horarias de entrega ofrecidas en cada zona.
3. Configuración del **cuadro tarifario**, con variación al menos por zona, peso, volumen y modalidad de servicio (estándar y urgente), contemplando recargos y bonificaciones. La configuración se realiza desde la interfaz, sin tocar código ni realizar un nuevo despliegue.
4. Configuración de las **reglas operativas** del operador: cantidad máxima de intentos de entrega, plazo entre intentos, tipo de prueba de entrega exigida, catálogo de motivos válidos de no entrega, política de devolución y plazos comprometidos por modalidad de servicio.
5. Gestión de la identidad visual del operador (marca, colores, datos de contacto), reflejada en el portal del comercio, en el seguimiento público y en las comunicaciones al destinatario.

6. Gestión de repartidores y vehículos, con su capacidad de carga.

5.2 Ciclo de vida del envío

7. Alta de envíos por parte del comercio: individual, por importación de archivo y por API. La importación masiva debe ser **idempotente**: reimportar un lote, completo o parcial, no puede generar envíos duplicados.
8. Un envío puede contener uno o más bultos, cada uno con su identificación, peso y dimensiones.
9. Cálculo automático de la tarifa al momento del alta, según el cuadro tarifario vigente del operador.
10. Recepción del envío en depósito mediante escaneo, con verificación contra lo declarado por el comercio y registro de discrepancias.
11. Gestión del estado del envío mediante una **máquina de estados explícita**. Las transiciones válidas deben estar definidas de forma declarativa y ser verificables; no se admite modificar el estado directamente desde cualquier punto del sistema. Los estados mínimos a contemplar son: admitido, en depósito, asignado a ruta, en tránsito, entregado, no entregado, reprogramado, en devolución, devuelto y extraviado.
12. Registro completo e inmutable de los eventos del envío, con marca temporal, origen, responsable y geolocalización cuando corresponda.

5.3 Planificación y despacho

13. Armado de hojas de ruta con asignación de envíos a repartidor y vehículo.
14. Validación de las restricciones de la ruta: cantidad máxima de paradas, capacidad de peso y volumen del vehículo, y compatibilidad con las franjas horarias comprometidas.
15. Garantía de que un mismo envío no pueda ser asignado simultáneamente a dos rutas por dos despachadores distintos.
16. Ordenamiento de las paradas de la ruta según un criterio razonable y documentado.
17. Despacho de la ruta y puesta a disposición de la aplicación del repartidor.

Aclaración de alcance. No se pide resolver el problema de optimización de rutas, sino la asignación con validación de restricciones y un ordenamiento razonable y justificado de las paradas. Los equipos que quieran abordar la optimización podrán hacerlo únicamente en el marco del requerimiento opcional correspondiente, y nunca en detrimento del resto.

5.4 Ejecución en la calle

18. Descarga de la hoja de ruta del día para su uso sin conectividad.
19. Escaneo de los bultos al cargar el vehículo, con verificación contra la hoja de ruta y aviso ante faltantes o sobrantes.
20. Registro de la entrega con **prueba de entrega**: firma del receptor en pantalla, fotografía, nombre y documento del receptor, según lo exija la configuración del operador. Cada prueba debe registrar la posición geográfica y la hora obtenidas del dispositivo.
21. Registro del intento fallido, con selección de motivo del catálogo configurado y su evidencia.
22. Gestión de la devolución y de la rendición al regresar al depósito.
23. Seguimiento de la posición del vehículo durante la jornada.
24. Operación completa sin conectividad y sincronización diferida al recuperarla, con **política explícita de resolución de conflictos** para los casos en que el estado registrado en el dispositivo y el del servidor sean incompatibles.

5.5 Seguimiento y comunicación

25. Seguimiento público del envío por parte del destinatario mediante enlace, sin autenticación y sin exponer datos personales de terceros ni identificadores internos del sistema.
26. Solicitud de reprogramación por parte del destinatario, sujeta a las reglas del operador.

27. Notificación automática al destinatario ante los cambios de estado relevantes.
28. **Avisos automáticos a los sistemas de los comercios.** Ante cada cambio de estado, la plataforma notifica al sistema del comercio mediante una llamada HTTP saliente, con las garantías de entrega que se detallan en la sección 6.
29. Tablero de operación en vivo, con la posición de la flota y el detalle de los envíos en riesgo de incumplir su ventana comprometida.
30. Reportes de gestión: cumplimiento por zona, repartidor y comercio; motivos de no entrega; volumen por período; y liquidación por comercio.

6. Requerimientos no funcionales obligatorios

Los requerimientos de esta sección son de cumplimiento obligatorio. Su ausencia, o una implementación meramente nominal, impacta directamente en la calificación del equipo.

6.1 Plataforma y arquitectura

La solución se desarrollará en .NET 10. La arquitectura de despliegue será la de un **monolito modular** para la API y las aplicaciones web, más un **servicio de procesamiento en segundo plano desplegado de forma independiente**, que se comunica con el resto exclusivamente de manera asíncrona a través de la cola de mensajes.

El estilo arquitectónico interno queda a criterio del equipo. Cualquiera sea la elección, deberán cumplirse tres condiciones verificables:

- La lógica de dominio no depende de la infraestructura, del acceso a datos ni de los frameworks de presentación.
- Esa regla de dependencias está respaldada por al menos una **prueba automatizada de arquitectura** (por ejemplo con NetArchTest o ArchUnit.NET) que se ejecuta en el pipeline y falla si la regla se viola.
- El estilo adoptado, con su organización de proyectos y carpetas, está justificado en un registro de decisión de arquitectura.

Todo el entorno deberá ejecutarse en contenedores Docker y ser reproducible con **docker compose**, incluyendo los servicios de soporte: base de datos, caché, mensajería y observabilidad.

6.2 Capa de presentación web

El backoffice del operador se desarrollará con ASP.NET Core Razor Pages. El portal del comercio y el seguimiento público se desarrollarán con **Blazor**. La elección del modo de renderizado (Server, WebAssembly o automático) queda a criterio del equipo, pero deberá respaldarse en un registro de decisión que analice sus consecuencias sobre el escalado horizontal, el manejo de estado, la latencia y el consumo de recursos del servidor.

6.3 Seguridad y autenticación

- Autenticación bajo OpenID Connect / OAuth 2.1 (Authorization Code con PKCE) o mediante ASP.NET Identity como almacén de usuarios.
- Autorización a nivel de API, contemplando los perfiles de la realidad y, además, el aislamiento por operador y por comercio.
- Secretos gestionados fuera del código fuente y fuera del repositorio.
- TLS, CORS restringido, limitación de tasa, cabeceras seguras y protección específica del seguimiento público frente a la enumeración de envíos.
- Cifrado en reposo de los datos personales almacenados en el dispositivo móvil.

6.4 Persistencia y datos

La base de datos principal queda a elección del equipo, administrada con Entity Framework Core y migraciones controladas. La aplicación móvil usará SQLite para su almacenamiento local. Se deberá implementar **control de concurrencia optimista** sobre las entidades susceptibles de

modificación simultánea, y demostrar su funcionamiento.

6.5 Multitenancy

- Estrategia de tenencia definida y justificada (por fila, por esquema o por base de datos), incluyendo el tratamiento del segundo nivel de aislamiento que corresponde a los comercios dentro de cada operador.
- Resolución del inquilino en tiempo de ejecución.
- Aislamiento efectivo en lecturas y escrituras, con pruebas automatizadas que validen que no hay filtración de datos entre inquilinos.
- Identidad visual y reglas de negocio diferenciadas por inquilino.
- Migraciones y datos de inicialización con soporte multi-inquilino.
- Demostración con al menos dos operadores con configuraciones distintas.

6.6 Configurabilidad y parametrización

El cuadro tarifario y las reglas operativas deberán poder modificarse desde la interfaz de administración, en tiempo de ejecución, sin modificar código ni realizar un nuevo despliegue. Los cambios de configuración deberán estar versionados: un envío procesado bajo una configuración anterior conserva su tratamiento original.

6.7 Caché distribuido

Se deberá implementar caché distribuido sobre al menos **dos casos de uso con perfiles de acceso diferentes**, con política de expiración e invalidación coherente con el dominio en cada caso, y con métrica de aciertos y fallos visible en el tablero técnico. A modo de sugerencia: el seguimiento público combina alta frecuencia de lectura con datos que cambian seguido, mientras que la configuración tarifaria combina alta frecuencia de lectura con datos que casi no cambian.

6.8 Mensajería asíncrona y procesamiento en segundo plano

- Publicación de eventos de negocio desde la API y procesamiento por el worker.
- **Patrón outbox**, de modo que no exista posibilidad de que un cambio se persista sin que su evento se publique, ni a la inversa.
- **Idempotencia** en el consumidor: procesar dos veces el mismo mensaje no puede producir efectos duplicados.
- Reintentos con espera creciente y cola de mensajes fallidos.
- Visibilidad operativa: el administrador debe poder consultar los mensajes fallidos y reintentarlos.

6.9 Avisos automáticos a los comercios

La entrega de avisos a los sistemas externos de los comercios deberá contemplar: suscripción configurable por comercio y por tipo de evento; firma del mensaje que permita al receptor verificar su origen; reintentos con espera creciente; cola de fallidos; y un panel donde el comercio consulte el historial de entregas y reenvíe las que fallaron. Cada equipo deberá proveer un receptor de prueba con el que demostrar el mecanismo, incluidos los escenarios de falla.

6.10 Comunicación en tiempo real

El tablero de operación se actualizará en tiempo real mediante SignalR. Los canales en tiempo real respetarán los mismos mecanismos de autenticación y autorización que el resto del sistema, incluido el aislamiento por inquilino, y deberán funcionar correctamente con la API ejecutándose en más de una instancia.

6.11 Aplicación móvil con .NET MAUI

La aplicación se desarrollará con .NET MAUI aplicando el patrón MVVM, y deberá implementar las siguientes capacidades, que se corresponden con requerimientos funcionales de la realidad:

- Seguimiento de la posición durante la jornada, con la aplicación en segundo plano y la pantalla

apagada.

- Escaneo de códigos de barras o QR para la carga y verificación de bultos.
- Captura de prueba de entrega: firma en pantalla, fotografía con compresión, y registro de posición y hora tomadas del dispositivo.
- Operación completa sin conectividad, con almacenamiento local cifrado y sincronización diferida con resolución de conflictos documentada.
- Validación biométrica para las operaciones que se definan como sensibles.
- Recepción de notificaciones con la aplicación cerrada, para reasignaciones urgentes.

6.12 Observabilidad

La instrumentación de la aplicación se realizará con **Serilog** para el registro estructurado y **OpenTelemetry** para métricas y trazas distribuidas. La herramienta de visualización y centralización queda a libre elección del equipo: Grafana con Prometheus, Seq, Jaeger, el panel de .NET Aspire o el servicio administrado del proveedor de nube son todas opciones válidas.

Se exige un identificador de correlación que permita seguir una operación de extremo a extremo, desde su origen en la aplicación móvil hasta su procesamiento en el worker, y al menos un tablero técnico que muestre tiempo medio de respuesta, percentiles 95 y 99 de latencia, tasa de errores, profundidad de la cola, sincronizaciones pendientes, avisos a comercios fallidos y relación de aciertos del caché.

6.13 Escalabilidad y balanceo de carga

La solución deberá ejecutarse en al menos dos instancias simultáneas de la API, con el tráfico distribuido por un balanceador, garantizando continuidad ante la caída de una instancia. Como mínimo: API sin estado en memoria; puntos de verificación de salud `/health`, `/ready` y `/live`; apagado controlado; protección de datos compartida entre instancias cuando se usen cookies o Blazor Server; y garantía de que las tareas programadas no se ejecuten por duplicado al haber varias instancias.

6.14 Despliegue e infraestructura como código

- Despliegue en un proveedor de nube pública, sobre **servidor remoto real**. Queda expresamente excluido el uso de túneles o de servicios que expongan un equipo local (por ejemplo ngrok o Cloudflare Tunnel), así como servir la aplicación desde el equipo de un integrante.
- El proveedor es de libre elección: AWS, Google Cloud, Azure, Oracle Cloud u otro.
- Acceso por HTTPS con certificado válido.
- Toda la infraestructura definida mediante **código** (Terraform, OpenTofu, AWS CDK, Pulumi o Bicep), con variables por ambiente, secretos fuera del código y las operaciones de planificación, aplicación y destrucción documentadas y ejecutables.
- Pipeline de integración y entrega continua que compile, ejecute las pruebas y despliegue.
- Control y evidencia del costo del ambiente. Se recomienda trabajar dentro de los niveles gratuitos y destruir el ambiente cuando no se use: poder reconstruirlo íntegramente con un comando es parte de lo que se evalúa.

6.15 Calidad

Pruebas unitarias sobre la lógica de dominio, pruebas de integración sobre al menos un flujo crítico completo, la prueba de arquitectura mencionada más arriba, las pruebas de aislamiento entre inquilinos, y la ejecución automática de todas ellas en el pipeline.

Equipos de 3 integrantes. Quedan eximidos del requerimiento de escalabilidad y balanceo de carga. La aplicación móvil puede reemplazarse por una aplicación web progresiva con operación sin conectividad. El mínimo de puntos opcionales exigido se reduce según la sección 7.

7. Requerimientos opcionales

A continuación se presenta un catálogo de requerimientos opcionales. Cada uno tiene asignada una cantidad de **puntos** según su dificultad y su volumen de trabajo. Estos puntos indican cuánto trabajo opcional cubre cada equipo y no guardan relación con la calificación del curso, que se rige por la sección 9.

El catálogo está deliberadamente sobredimensionado: no se espera que un equipo aborde todo, sino que arme su propio recorrido según sus intereses y afinidades.

Mínimo a cubrir: 12 puntos para equipos de 4 integrantes y 8 puntos para equipos de 3. Un requerimiento implementado parcialmente no otorga puntos parciales: se evalúa contra las condiciones indicadas en cada caso.

7.1 Arquitectura y comunicación

Pts	Requerimiento
4	Extracción del seguimiento público como servicio independiente. Desplegarlo como una aplicación separada, con su propia base de datos de solo lectura, alimentada exclusivamente por los eventos que circulan por la cola. Se exige base de datos propia sin acceso a la transaccional, consistencia eventual documentada y medida, despliegue y escalado independientes, y comportamiento definido ante atraso o caída del consumidor.
4	gRPC como canal de sincronización y telemetría de la aplicación móvil. Reemplazar o complementar el canal REST/JSON entre la aplicación MAUI y el backend, usando <i>client streaming</i> para el envío de lotes de eventos. Se exigen contratos en Protocol Buffers y evidencia medida que compare ambos canales sobre el mismo conjunto de datos, en bytes transferidos, tiempo de sincronización y cantidad de conexiones, más el análisis del comportamiento en red degradada. No sustituye a SignalR para los clientes web.
3	Apache Kafka para el flujo de eventos de la flota. Incorporar Kafka para la telemetría de posición y los eventos de envío, con más de un consumidor independiente del mismo flujo, y demostrar la reconstrucción completa de un estado derivado mediante la reproducción del log. Debe justificarse su coexistencia con la cola de trabajo obligatoria y contemplarse su costo en recursos.
3	Autoescalado horizontal. Escalado automático por métricas de carga, con evidencia del comportamiento del sistema durante una prueba de carga.

7.2 Datos y rendimiento

Pts	Requerimiento
2	Almacenamiento NoSQL para eventos históricos y analítica. Base documental para el historial de eventos y las consultas analíticas, sin afectar la base transaccional, con índices adecuados, política de retención y API de lectura paginada y filtrable.

Pts	Requerimiento
2	Pruebas de carga automatizadas. Plan con escenarios representativos (conurrencia, rampa, duración), objetivos de nivel de servicio definidos de antemano y ejecución sobre el ambiente remoto. Debe incluir el escenario de sincronización simultánea de varios repartidores al regresar al depósito.
2	Cobertura de pruebas. Cobertura verificable superior al 70 % en las capas de dominio y aplicación, con reporte publicado por el pipeline.
2	Pruebas de extremo a extremo automatizadas. Playwright o equivalente sobre los flujos críticos, integradas al pipeline.

7.3 Movilidad y terreno

Pts	Requerimiento
3	Notificaciones remotas en producción. Notificaciones push reales recibidas con la aplicación cerrada, aplicadas a la reasignación urgente de un envío, con gestión del ciclo de vida de los identificadores de dispositivo.
3	Geocercas. Detección automática de llegada y salida de parada, con el registro de eventos correspondiente y análisis del impacto sobre el consumo de batería.
2	Impresión por Bluetooth. Impresión de etiqueta o comprobante de entrega desde la aplicación móvil hacia una impresora térmica portátil.
2	Optimización del orden de paradas. Ordenamiento de la ruta mediante un algoritmo o servicio de optimización, con comparación medida contra el criterio adoptado en el requerimiento funcional 16.

7.4 Operación, seguridad y experiencia

Pts	Requerimiento
3	API pública para comercios. API documentada y versionada para la integración de los comercios, con claves por comercio, ambiente de pruebas, limitación de tasa diferenciada y documentación navegable.
2	Banderas de funcionalidad. Activación y desactivación de funcionalidades por inquilino en tiempo de ejecución, sin nuevo despliegue.
2	Pruebas de resiliencia. Interrumpir deliberadamente dependencias (caché, cola, base de datos, servicio externo) y documentar el comportamiento degradado esperado y el observado.

8. Entregas y cronograma

8.1 Entrega de análisis y diseño

Vence el **domingo 4 de octubre a las 23:59 h** por el Moodle del curso. Cada equipo deberá presentar el diagrama de arquitectura lógica y el de despliegue previsto; el modelo de dominio preliminar con las entidades clave y sus relaciones; el diagrama de la máquina de estados del envío con sus transiciones válidas; la estrategia de multitenancy elegida y su justificación; la propuesta de stack tecnológico y entorno de desarrollo; el plan de trabajo con la asignación de responsabilidades por integrante; y los dos primeros registros de decisión de arquitectura.

8.2 Registros de decisión de arquitectura

Cada equipo deberá entregar a lo largo del laboratorio un conjunto de **registros de decisión de arquitectura**, cada uno con un integrante responsable identificado. Cada registro es un documento breve, con cuatro partes: el contexto y las restricciones; la decisión adoptada; las alternativas consideradas y el motivo de su descarte; y las consecuencias, incluido aquello que la decisión dificulta a futuro.

Son de tratamiento obligatorio, al menos, las siguientes decisiones: el estilo arquitectónico y la organización del código; la estrategia de multitenancy; el modo de renderizado de Blazor; la política de resolución de conflictos de sincronización; la estrategia de caché e invalidación; y la garantía de consistencia entre la persistencia y la publicación de eventos.

8.3 Cronograma de monitoreos

Cada monitoreo exige demostrar en ejecución el objetivo comprometido.

Fecha	Objetivo verificable
8/10	Esqueleto ejecutable de extremo a extremo: <code>docker compose up</code> levanta el entorno completo, un envío dado de alta desde el portal del comercio se ve en el backoffice, y el pipeline de integración continua corre.
15/10	Máquina de estados operativa; multitenancy funcionando con dos operadores de configuración distinta; autenticación y autorización por perfil.
22/10	Aplicación móvil descargando la hoja de ruta, registrando entregas sin conectividad y sincronizando; caché en funcionamiento con su métrica de aciertos.
29/10	Mensajería asíncrona con worker independiente; avisos a comercios con reintentos y cola de fallidos; tablero en tiempo real. En esta instancia cada equipo recibirá un cambio de requerimientos no adelantado, de incorporación obligatoria.
5/11	Solución desplegada en la nube mediante infraestructura como código, con al menos dos instancias balanceadas y el tablero de observabilidad operativo.
12/11	Requerimientos opcionales completos; ensayo de la presentación y de la demo; revisión final previa a la entrega.

Sobre el cambio de requerimientos del 29 de octubre. Se anuncia desde ahora que existirá, pero no cuál será. Su propósito es evaluar la capacidad de la arquitectura construida para absorber un cambio, que es la situación normal de cualquier proyecto real. Conviene tenerlo en cuenta al diseñar: una solución acoplada, con lógica de negocio dispersa o con reglas escritas directamente en el código, va a tener dificultades severas para incorporarlo en el tiempo que quede.

8.4 Entrega final

Vence el **domingo 15 de noviembre a las 23:59 h** por el Moodle del curso, e incluye:

- Documento técnico que describa la implementación de los requerimientos no funcionales, las decisiones de diseño adoptadas, las configuraciones relevantes y los diagramas correspondientes.
- El conjunto completo de registros de decisión de arquitectura.
- La bitácora de uso de asistentes de IA.
- El registro de horas por integrante y por tipo de actividad.
- La presentación a utilizar en la defensa.

- El código fuente y el acceso al repositorio, con README actualizado, instrucciones de ejecución local y de despliegue remoto, y los scripts correspondientes.

9. Evaluación

9.1 Rúbrica

Componente	Peso
Requerimientos no funcionales obligatorios	30 %
Requerimientos funcionales	20 %
Presentación y demostración en la defensa	15 %
Requerimientos opcionales cubiertos	10 %
Documentación final	10 %
Análisis y diseño (entrega del 4 de octubre)	8 %
Registros de decisión de arquitectura	7 %
Total	100 %

La **calificación individual** de cada integrante puede apartarse de la calificación del equipo, según su desempeño en los monitoreos, los registros de decisión bajo su responsabilidad, su contribución verificable en el repositorio y su desempeño en la defensa.

9.2 Presentación y demostración

La defensa incluye una presentación formal del trabajo, con soporte visual, seguida de una demostración del sistema en funcionamiento. Deberán exponer **todos los integrantes** del equipo. Se evaluará tanto el contenido técnico como la calidad de la exposición: claridad, manejo del tiempo, capacidad de mostrar el producto en vivo y de responder preguntas.

9.3 Escenarios a demostrar

Durante la defensa se ejecutarán escenarios sobre el sistema desplegado. Se anuncian de antemano los siguientes:

1. Interrumpir una instancia de la API mientras el sistema recibe tráfico y verificar la continuidad del servicio.
2. Operar la aplicación móvil sin conectividad, generar entregas e incidencias, y restablecer la conexión.
3. Provocar un conflicto de sincronización y observar la aplicación de la política definida.
4. Operar en simultáneo con dos operadores de configuración distinta y verificar el aislamiento.
5. Modificar una tarifa y una regla operativa en caliente, y verificar su efecto.
6. Detener el receptor de avisos de un comercio, generar eventos, reactivarlo y verificar la recuperación de los mensajes.
7. Destruir y reconstruir el ambiente mediante infraestructura como código.

Se plantearán además **escenarios no anunciados** y se pedirá incorporar una **modificación menor en el momento**, resuelta frente al docente. Lo que se busca constatar no es la dificultad de esa modificación, sino que el equipo conozca su propia arquitectura y sepa dónde interviene cada componente.

9.4 Defensa individual del trabajo

A cada integrante se le pedirá explicar componentes y fragmentos de código de su área de responsabilidad, seleccionados por el docente. Se valorará la comprensión del funcionamiento, de las alternativas que se consideraron y de las consecuencias de lo implementado.

9.5 Bitácora de uso de asistentes de IA

Cada equipo deberá mantener y entregar una bitácora en la que, por componente relevante del sistema, se declare si se usó asistencia de IA, con qué finalidad, qué se aceptó sin modificaciones, qué se modificó y por qué, y qué se descartó. Declarar el uso de la herramienta es parte de la práctica profesional y no tiene consecuencia negativa alguna; omitirlo o falsearlo, en cambio, sí constituye una falta.

Criterio rector. Se evalúa la comprensión demostrada, no el volumen de código entregado. Un sistema con menos funcionalidades, íntegramente comprendido, fundamentado y defendible, obtendrá mejor calificación que un sistema extenso que sus autores no puedan explicar.

9.6 Condiciones de aprobación

Para aprobar el laboratorio se requiere cubrir con los requerimientos mínimos, tanto funcionales como no funcionales, cubrir el mínimo de los opcionales, y alcanzar un 60 % del total de la rúbrica. Por supuesto, también haber asistido a los monitoreos con avances verificables, y aprobar la defensa individual.